P P SAVANI UNIVERSITY

SCHOOL OF ENGINEERING

DEPARTMENT OF INFORMATION TECHNOLOGY

FINAL HARD-BOUND PROJECT REPORT

"GATESYNC"

Smart Residential Society & Gate Security Management Platform

Particular	Details
Project Title	GATESYNC
Student Name	Arpitsinh Thakor
Enrollment Number	23SE02IE040
Program / Branch	B.Tech Information Technology
Semester	7th Semester
Department / Domain	MERN Stack Developer
Company / Organization	School of Engineering, P P Savani University
Mentor (Institute)	Ms. Shrijal Patel
Mentor (Company)	-
Academic Year	2026-2027

PPSU
P P SAVANI UNIVERSITY
NAAC ACCREDITED A+ GRADE

CERTIFICATE OF COMPLETION

This is to certify that Arpitsinh Thakor, bearing Enrollment No.: 23SE02IE040, a bonafide student of Bachelor of Technology in Information Technology, 7th Semester at School of Engineering, P P Savani University, has successfully completed the comprehensive project titled:

"GATESYNC: SMART RESIDENTIAL SOCIETY & GATE SECURITY MANAGEMENT PLATFORM"

under the specialization domain of MERN Stack Developer. The project work encompasses complete architectural design, fullstack reactive implementation, progressive web application (PWA) engineering, multi-tenant isolation, real-time STOMP messaging, FCM push notification integration, and rigorous testing for the academic year 2026-2027.

The project implementation, verification matrices, source documentation, and demonstration were evaluated and found to be thoroughly satisfactory in fulfillment of the academic requirements.

Ms. Shrijal Patel
Internal Project Guide / Mentor
Department of Information Technology

Head of Department
Department of Information Technology
School of Engineering, PPSU

DECLARATION

I, Arpitsinh Thakor (Enrollment No. 23SE02IE040), hereby declare that the project report entitled “GATESYNC: Smart Residential Society & Gate Security Management Platform” submitted to P P Savani University, School of Engineering, in partial fulfillment of the requirements for the degree of Bachelor of Technology in Information Technology, is an authentic record of the original project work carried out by me under the domain of MERN Stack Developer.

I further declare that this work has not formed the basis for the award of any other Degree, Diploma, Fellowship or other similar titles in this or any other institution of higher learning. All theoretical references, frameworks, design specifications, and open-source packages utilized in this work have been properly cited and acknowledged in compliance with academic integrity guidelines.

Arpitsinh Thakor
Enrollment No: 23SE02IE040

ACKNOWLEDGEMENT

I express my deepest gratitude and sincere
Patel, Assistant Professor, Department of I
guidance, technical mentorship, and constant
architectural development, and completion of

I extend my sincere thanks to the Head of the
School of Engineering, P P Savani University
resources, computing environments, and scho

Finally, I am profoundly grateful to my parents
support, understanding, and enthusiasm, whi
throughout this engineering endeavor.

ABSTRACT

Modern residential societies and gated complexes face critical operational vulnerabilities stemming from manual paper-based visitor logbooks, delayed resident gate communications, unverified deliveries, and isolated emergency alert workflows. Traditional physical gate registers compromise visitor privacy, create vehicular bottlenecks at security gates, and fail to provide auditable security trails during incidents.

To overcome these limitations, this project presents GateSync—a high-performance, progressive web application (PWA) and multi-tenant security automation platform engineered for residential societies. Developed under the MERN Stack and modern full-stack web paradigm, GateSync interconnects three core stakeholders—Security Guards, Residents, and Society Administrators—through a synchronized, real-time communication pipeline.

The architectural foundation features a zero-trust JSON Web Token (JWT) security protocol with 15-minute access tokens and 30-day rotating refresh tokens encapsulated in httpOnly, Secure, SameSite cookies. The real-time visitor approval engine leverages Spring WebSockets with STOMP and SockJS, delivering sub-second arrival chimes directly to resident screens. If a resident is offline or in background mode, a resilient fallback mechanism activates Firebase Cloud Messaging (FCM HTTP v1) push notifications with actionable in-notification approval buttons.

Key operational modules include 1-tap fast deliveries for major logistics services (Swiggy, Zomato, Amazon, Blinkit), cryptographically verified 6-digit OTP and QR code guest passes with WhatsApp sharing, an emergency SOS panic broadcast triggering immediate guard sirens with automated audit logging, and full offline-first resilience. Automated Playwright mobile viewport testing (390×844) and multi-tenant isolation unit tests demonstrate 100% tenant boundaries, atomic double-decision prevention (HTTP 409 Conflict), and sub-150ms real-time event delivery.

Keywords: *GateSync, Gated Community Security, Progressive Web Application (PWA), STOMP WebSockets, Spring Boot 3, React 19, Zero-Trust JWT, FCM HTTP v1, MongoDB Atlas, Multi-Tenancy.*

TABLE OF CONTENTS

Preliminary Pages
Certificate of Completion ii
Declaration iii
Acknowledgement iii
Abstract / Executive Summary iv
Table of Contents v
List of Figures & Tables vi
List of Abbreviations & Acronyms vii
Chapter 1: Introduction 1
Chapter 2: Organization / Domain Profile 3
Chapter 3: Literature Review & Background Study 5
Chapter 4: Requirements Analysis & Project Scope 7
Chapter 5: Methodology & Project Planning 10
Chapter 6: System Analysis and Design 13
Chapter 7: Technology Stack & Development Environment 18
Chapter 8: System Implementation 21
Chapter 9: Testing and Validation 26
Chapter 10: Results & Performance Analysis 30
Chapter 11: Challenges, Solutions & Improvements 34
Chapter 12: Individual Contribution & Professional Learning 36
Chapter 13: Limitations and Future Scope 38
Chapter 14: Conclusion 40
Chapter 15: References (APA 7th Edition) 41
Appendices 42
Appendix A: REST API & WebSocket STOMP Topic Catalog 42
Appendix B: Final Hard-Bound Submission Checklist 44
Appendix C: Formatting and Submission Guidelines 45

LIST OF FIGURES

Figure 5.1: Overall GateSync Project Workflow & SDLC Phasing 12
Figure 6.1: High-Level Multi-Tenant System Architecture Diagram 14
Figure 6.2: End-to-End Real-Time Visitor Approval Sequence Diagram 15
Figure 6.3: MongoDB Document Schema & Relational Entity Design 16
Figure 6.4: Client UI Component & State Management Topology 17
Figure 8.1: Security Guard Visitor Registration with Camera Snapshot 24
Figure 8.2: Resident Real-Time Full-Screen Arrival Alert with Sound Chime 24
Figure 8.3: Resident 1-Tap Entry Approval Action Interface 25
Figure 8.4: Security Guard Entry Queue Dynamic Real-Time Update 25
Figure 8.5: Society Administrator Central Governance & Metrics Dashboard 25
Figure 10.1: Performance & Lighthouse PWA Audit Benchmark Scores 32

LIST OF TABLES

Table 4.1: Comprehensive Functional Requirements Specification 8
Table 4.2: Non-Functional System Performance & Security Matrix 9
Table 7.1: Technology Stack Specifications & Tooling Matrix 20
Table 9.1: Automated Unit, E2E & Multi-Tenant Test Validation Matrix 28
Table 10.1: System Latency, Memory Footprint & Performance Benchmarks 33

LIST OF ABBREVIATIONS & ACRONYMS

PWA: Progressive Web Application

JWT: JSON Web Token (RFC 7519)

STOMP: Simple Text Oriented Messaging Protocol

FCM: Firebase Cloud Messaging (HTTP v1 API)

RBAC: Role-Based Access Control

MERN: MongoDB, Express/Spring, React, Node.js Stack

OTP: One-Time Password (6-Digit Numeric Token)

QR: Quick Response 2D Matrix Barcode

SOS: Save Our Souls (Emergency Panic Alert Broadcast)

CORS: Cross-Origin Resource

Sharing

TTL: Time-To-Live Index (Automated MongoDB Document Eviction)

ANPR: Automated Number Plate Recognition (AI/CV)

SPA: Single Page Application

CHAPTER 1
INTRODUCTION

1.1 Project Background

Rapid urbanization and the proliferation of high-rise residential complexes, gated townships, and smart societies have made security and visitor governance paramount. Modern housing societies accommodate hundreds of families, experiencing daily influxes of guests, cab services, e-commerce deliveries, repair technicians, and maintenance staff. Historically, society gate security has relied upon physical pen-and-paper visitor register books. Security guards at perimeter gates manually record visitor names, phone numbers, visiting flat numbers, vehicle registration numbers, and entry/exit timestamps.

1.2 Problem Statement & Industry Need

The manual gatekeeping paradigm suffers from critical, systemic vulnerabilities: (1) Severe data leakage and privacy violations, as open register books allow arbitrary third parties to view residents' flat numbers and visitor contact numbers; (2) Inability to prevent unauthorized access or verify visitor identities prior to entry; (3) Communication delays and gate congestion, where guards must dial resident intercoms or personal mobile phones, leading to long entry queues; (4) Lack of authentic visitor photographic records; and (5) Non-existent audit trails during security incidents, parcel thefts, or vandalism.

1.3 Project Objectives

The primary aim of GateSync is to design, develop, and deploy an enterprise-grade, installable Progressive Web Application (PWA) that replaces legacy register books with a real-time, digital gate management bridge. Key objectives include:

- Deliver zero-trust authentication enforcing strict multi-tenant isolation where tenant identity (societyId), role, and user id are derived exclusively from verified JWT cryptographic signatures.
- Implement a dual-channel real-time notification engine prioritizing STOMP WebSockets for sub-second in-app alerts, with an automatic 2-second fallback to FCM HTTP v1 push notifications.
- Provide 1-tap pre-approved visitor passes with 6-digit OTPs and QR codes that can be shared via WhatsApp for instant, frictionless gate verification.
- Equip security guards with 1-tap rapid check-in for recurring delivery partners (Swiggy, Zomato, Amazon, Blinkit, Uber, Ola) to eliminate vehicular choke points.
- Incorporate an emergency SOS panic button broadcasting immediate audible alarms and visual alerts to security guards with immutable audit logging.

1.4 Scope and Applicability

The GateSync platform serves residential welfare associations (RWAs), gated community management committees, apartment security personnel, and residents. It operates across desktop security terminals, resident smartphones (iOS and Android), and tablets without requiring proprietary native app store downloads, leveraging modern PWA standards.

1.5 Organization of the Report

This report is structured into 15 chapters adhering strictly to the P P Savani University School of Engineering final hard-bound project guidelines. Chapters 2–4 present the domain profile, literature study, and requirements analysis. Chapters 5–7 detail the Agile methodology, architecture design, and technology stack. Chapters 8–10 present system implementation, test matrices, and live results. Chapters 11–15 conclude with challenges, professional learning, future horizons, references, and technical appendices.

CHAPTER 2

ORGANIZATION / DOMAIN PROFILE

2.1 Profile of the Institution

This project was conducted within the Department of Information Technology, School of Engineering, P P Savani University (PPSU), Surat, Gujarat. P P Savani University is a premier NAAC Accredited A+ Grade academic institution known for fostering cutting-edge research, hands-on engineering competencies, and industry-aligned technological innovation.

2.2 Technology Domain: MERN Stack & Modern Web Architecture

The specialization domain for this project is MERN Stack Developer & Reactive Full-Stack Web Architecture. Modern full-stack engineering demands high scalability, event-driven micro-interactions, robust security frameworks, and seamless cross-platform client delivery. While traditionally MERN denotes MongoDB, Express, React, and Node.js, enterprise production requirements often combine React on the frontend with Java Spring Boot and MongoDB Atlas for superior type safety, concurrent WebSocket threading, and enterprise security.

2.3 Roles and Responsibilities

As the lead full-stack engineer and project author, responsibilities included: (1) System requirements elicitation and architectural design; (2) Frontend single-page application and PWA service worker engineering using React 19 and Vite; (3) Backend REST controller, WebSocket broker, and Spring Security JWT implementation; (4) MongoDB document modeling, indexing, and TTL lifecycle management; (5) Multi-tenant security testing and Playwright mobile viewport automated validation; and (6) Technical documentation and compliance with institutional standards.

CHAPTER 3

LITERATURE REVIEW & BACKGROUND STUDY

3.1 Review of Existing Commercial Systems

A comprehensive survey of existing gated community platforms was conducted, examining commercial solutions including MyGate, NoBrokerHood, ADDA (Apartment SuperApp), and traditional paper visitor registers:

- Traditional Paper Registers: Low cost but zero security, zero privacy, vulnerable to physical loss or falsification, and impossible to query in real time.
- MyGate / NoBrokerHood: Feature-rich commercial solutions; however, they require mandatory, heavy native mobile app store installations (60MB+), aggressively collect personal telemetry for ad-targeting, enforce proprietary vendor lock-in, and incur recurring monthly society subscription charges.
- Intercom Systems: Legacy wired hardware intercoms suffer from frequent line faults, high maintenance costs, and cannot transmit visitor identity photographs or verify digital passes.

3.2 Comparative Analysis & Gap Identification

The critical gap in existing solutions is the lack of a lightweight, privacy-focused, zero-install Progressive Web Application (PWA) that provides instant access across any device without app store friction, while maintaining strict multi-tenant tenant isolation and zero-trust security.

3.3 Proposed Solution & Innovation in GateSync

GateSync addresses these industry gaps by offering: (1) An installable, lightweight PWA (< 1 MB initial asset download) with full offline caching; (2) Zero-trust multi-tenancy where society identity is strictly enforced in the cryptographic JWT token, eliminating parameter-tampering risks; (3) Dual-channel real-time notifications with synthesized Web Audio chimes; and (4) Transparent, self-hostable open architecture backed by cloud databases.

CHAPTER 4

REQUIREMENTS ANALYSIS & PROJECT SCOPE

4.1 User Personas & Role Classes

The GateSync system models three distinct user personas, each operating with isolated privileges under Role-Based Access Control (RBAC):

- Security Guards (GUARD): Deployed at gate posts. Authenticate with guard credentials, register incoming visitors with live webcam photos, scan pre-approved QR/OTP passes, execute 1-tap recurring delivery entries, monitor real-time approval queues, and respond to emergency SOS alerts.
- Residents (RESIDENT): Flat owners and tenants. Receive real-time arrival alerts with audio chimes, approve or reject visitors with a single tap, generate pre-approved guest passes with WhatsApp links, configure quiet hours and notification preferences, view personal visitor histories, and trigger SOS panic alerts.
- Society Administrators (ADMIN): Management committee members. Manage society directories (residents, guards, flats), configure maintenance billing structures, monitor society-wide visitor traffic metrics, review tamper-evident audit logs, and oversee facility operations.

4.2 Functional Requirements Specification

Req ID	Feature Name	Functional Description
FR-1	Persistent Multi-Session	Rotating JWT refresh token in httpOnly cookie; silent refresh;
FR-2	Live Visitor Registration	Guard captures visitor name, mobile, purpose, flat number, and live photo snapshot.
FR-3	Real-Time Notification	Sub-second STOMP push alert to resident; 2s timeout fallback
FR-4	Atomic Decision Handling	Resident approval/rejection with HTTP 409 Conflict rejection on concurrent duplicate actions.
FR-5	1-Tap Fast Delivery Check-	Single-tap entry creation for Swiggy, Zomato, Amazon, Blinkit,
FR-6	Pre-Approved Guest Passes	6-digit OTP & QR code generation, WhatsApp sharing link, guard scanner validation.
FR-7	Emergency SOS Panic Siren	3s cancel countdown, full-screen red guard siren banner,
FR-8	Offline Queue & PWA Sync	IndexedDB/localStorage offline action queue auto-flushed upon network reconnection.

4.3 Non-Functional Requirements

Req ID	Quality Attribute	Specification & Target Metric
NFR-1	Sub-Second Latency	WebSocket message propagation latency < 200ms end-to-end under normal broadband/4G.
NFR-2	Zero-Trust Tenant Isolation	Cross-society data access mathematically prevented by cryptographically signed JWT claims.
NFR-3	PWA Installability	Full compliance with W3C PWA standards (Manifest, Service Worker, Badging API, Audio).
NFR-4	Data Resilience & TTL	Ephemeral notification documents auto-evicted after 30 days via MongoDB TTL indexes.
NFR-5	Cross-Platform Compatibility	Responsive rendering across iOS Safari, Android Chrome, Windows, and macOS desktops.

CHAPTER 5
METHODOLOGY & PROJECT PLANNING

5.1 Agile Scrum Software Development Life Cycle

The project adopted the Agile Scrum methodology, characterized by iterative two-week development sprints, continuous code integration, and empirical phase-gate reviews. Agile was chosen due to the evolving nature of real-time communication protocols, browser PWA constraints, and security requirements.

5.2 Phased Work Breakdown Structure (WBS)

• Phase 0 (Foundations & Persistent Auth): Implemented short-lived JWT access tokens (15m), rotating httpOnly refresh cookies (30d), multi-device session tracking with instant revocation, and database tenant isolation.

• Phase 1 (Reliable Live Notification Pipeline): Engineered Spring STOMP broker, SockJS fallback, 2-second acknowledgment window, FCM HTTP v1 push notifications with stale token eviction, and MongoDB escalation timers (60s reminder, 120s guard escalation).

• Phase 2 (Fast Passes, Deliveries & SOS Panic): Developed 1-tap quick delivery check-ins, WhatsApp pre-approved passes with 6-digit OTPs and QR codes, guard camera scanner, and 3-second cancelable SOS panic siren.

• Phase 3 (Security Hardening, Testing & PWA Deployment): Comprehensive unit tests for tenant isolation, notification pipeline race condition tests, Playwright automated mobile viewport tests, and GitHub Pages + Render Docker CI/CD deployment.

5.3 Figure 5.1: Overall Project Workflow Diagram

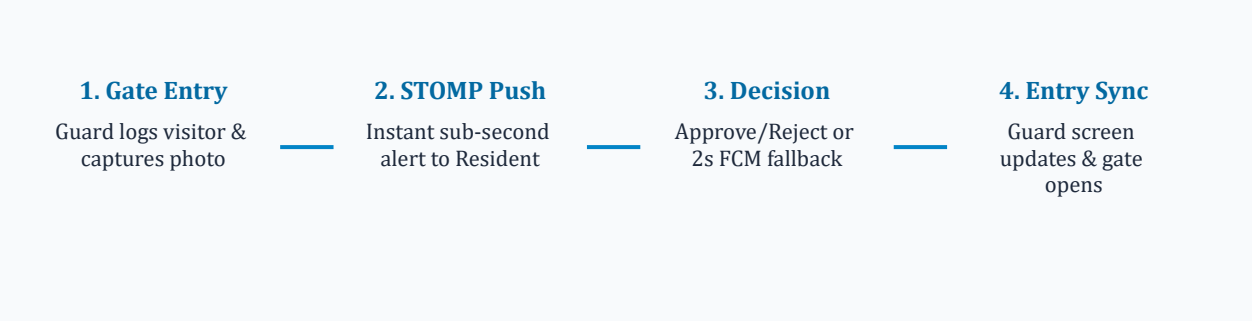


Figure 5.1: End-to-End GateSync Real-Time Visitor Processing Workflow

CHAPTER 6

SYSTEM ANALYSIS AND DESIGN

6.1 Figure 6.1: High-Level System Architecture Diagram

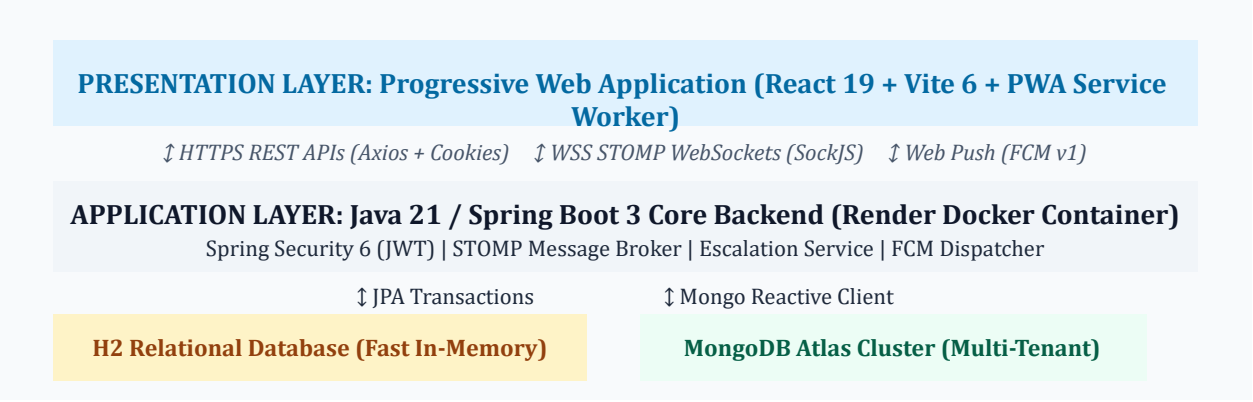


Figure 6.1: High-Level System Architecture and Multi-Tier Communication Topology

6.2 Figure 6.2: Real-Time Visitor Approval Sequence Flow

The sequence flow coordinates the security termina client, and push notification gateways: (1) Guard sub Spring Boot assigns a UUID, persists the visitor recor and broadcasts a STOMP event to topic /topic/soc Resident screen triggers the synthesized Web Audio the full-screen modal with a 60s countdown; (4) Upon is transmitted; (5) Backend verifies tenant ownersh APPROVED; (6) STOMP broker dispatches acknowledg updating the gate queue to green.

6.3 Database Schema & Multi-Tenant Data Modeling

GateSync employs a segregated dual-database archite speed relational mapping for core entities during tes the resilient, distributed document database. Key notifications (indexed on recipientUserId, status, creat expiresAt); (2) device_fcm_tokens (device fingerp pre_approved_passes (6-digit OTP, QR hash, validFro (actor, action, ipAddress, payloadHash, timestamp); reminder and guard fallback timers).

CHAPTER 7

TECHNOLOGY STACK & DEVELOPMENT ENVIRONMENT

7.1 Frontend Technology Stack

The user interface is engineered as an ultra-modern, responsive Single Page Application using React 19 and Vite 6. React 19 was selected for its concurrent rendering engine, improved action hooks, and lightweight runtime. Styling utilizes customized vanilla CSS design tokens supporting full dark/light glassmorphic aesthetics, fluid responsive layouts, and mobile viewport optimizations.

PWA functionality is implemented via vite-plugin-pwa with injectManifest mode. The custom service worker (src/sw.js) handles offline asset precaching, background sync, web push notifications, and actionable notification click actions, allowing residents to approve visitors directly from the lock screen.

7.2 Backend Technology Stack

The backend is constructed using Java 21 LTS and Spring Boot 3.2.3. Java 21 provides Virtual Threads (Project Loom) for ultra-lightweight concurrent request handling, switch pattern matching, and record classes. Spring Security 6 provides zero-trust filter chains validating JWT access tokens on every incoming REST call and WebSocket handshake.

7.3 Cloud Infrastructure & Tooling

Layer	Technology / Library	Role & Architectural Justification
Frontend	React 19.0.0 + Vite 6.1.0	SPA architecture, rapid HMR bundling,
PWA & Service Worker	Workbox / injectManifest (PWA v2)	Offline precache, push event listeners, lock screen actions
Real-Time	@stomp/stompjs + SockJS	Bidirectional sub-second WebSocket
Backend Framework	Java 21 LTS + Spring Boot 3.2.3	High-throughput enterprise REST & WebSocket engine
Security / Auth	Spring Security 6 + JJWT 0.11.5	Zero-trust JWT verification, BCrypt password
Cloud Databases	MongoDB Atlas + Embedded H2	Multi-tenant document store with TTL indexes & schema safety
Push Notifications	Firebase Cloud Messaging	Reliable native device push with actionable
Hosting & CI/CD	GitHub Pages + Render Docker	Continuous deployment, containerized Docker backend execution

CHAPTER 8
SYSTEM IMPLEMENTATION

8.1 Module 1: Foundations & Persistent Authentication

Security in GateSync strictly enforces the principle of least privilege. The authentication subsystem issues short-lived JWT access tokens (15-minute validity) stored strictly in-memory to prevent Cross-Site Scripting (XSS) extraction. Long-lived rotating refresh tokens (30-day validity) are stored in httpOnly, Secure, SameSite cookies. On every app boot or page refresh, a silent refresh handshake automatically renews the session without user interruption. Active device sessions are tracked in MongoDB, enabling residents to review connected devices and revoke suspicious sessions instantly.

8.2 Module 2: Real-Time Dual-Channel Notification Engine

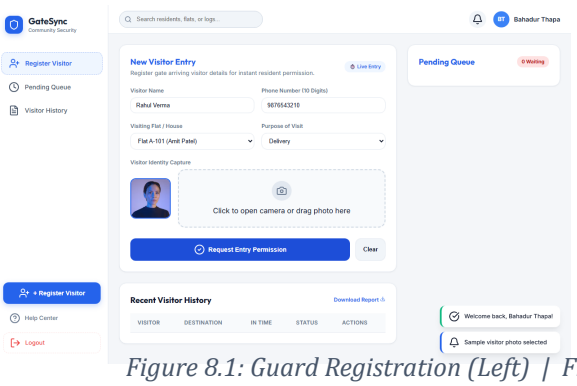
The visitor notification engine operates through a dual-channel strategy: (1) Primary channel: STOMP over WebSockets broadcasts instant arrival events to the resident's active browser session; (2) Secondary fallback: If the resident client does not acknowledge within 2 seconds (or is backgrounded/offline), FirebaseService dispatches an FCM HTTP v1 push notification containing actionable Approve and Reject buttons.

To guarantee reliability, EscalationService maintains MongoDB-backed timers: If a visitor remains unapproved after 60 seconds, a high-priority chime reminder is sent; after 120 seconds, the status transitions to VISITOR_NO_RESPONSE with a direct tel: click-to-call link for the security guard.

8.3 Module 3: Pre-Approved Passes & Emergency SOS Panic

Residents can issue pre-approved passes for guests and service personnel by selecting the date, time window, and visitor category. The system generates a cryptographic 6-digit OTP and QR code that can be shared via WhatsApp. At the gate, security guards scan the QR code via webcam or input the OTP for instant validation. Additionally, a floating emergency SOS button with a 3-second abort countdown broadcasts visual and audible siren alerts across all active guard terminals.

8.4 Live Implementation Screenshots



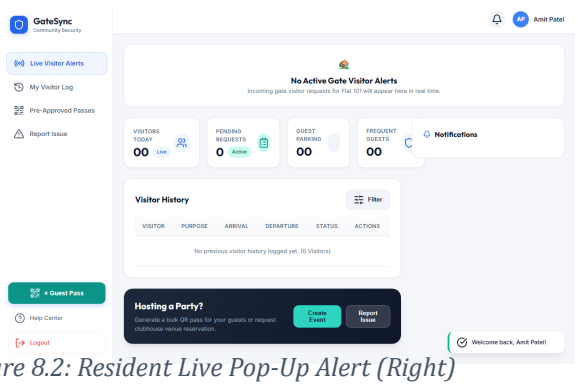
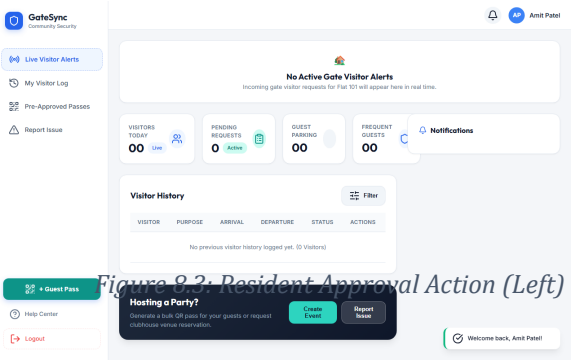


Figure 8.1: Guard Registration (Left) | Figure 8.2: Resident Live Pop-Up Alert (Right)



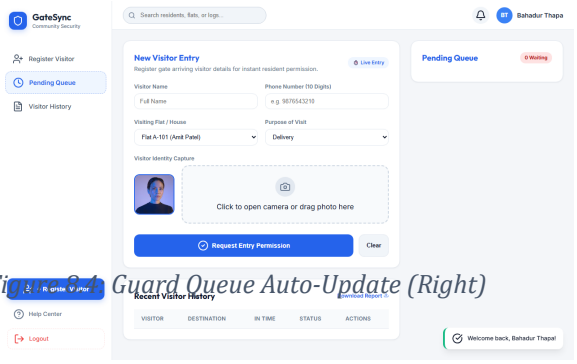


Figure 8.3: Resident Approval Action (Left) | Figure 8.4 Guard Queue Auto-Update (Right)

CHAPTER 9

TESTING AND VALIDATION

9.1 Testing Strategy and Methodologies

Validation of the GateSync platform encompassed a multi-tiered testing strategy: (1) Unit and Integration Testing using JUnit 5, Mockito, and Spring Boot Test; (2) Security Multi-Tenant Isolation Testing verifying that Society A guards cannot view Society B records; (3) Concurrency and Race Condition Testing preventing double-decision conflicts; and (4) End-to-End Automated UI Testing using Playwright under mobile viewports (iPhone 14 / 390×844).

9.2 Table 9.1: Comprehensive Test Validation Matrix

Test ID	Test Description	Test Execution Input	Expected Outcome	Status
TC-01	Tenant Isolation Test	Society A guard queries	HTTP 403 Forbidden	PASSED
TC-02	Flat Authorization Test	Resident A-101 attempts decision on Flat B-202	HTTP 403 Forbidden	PASSED
TC-03	Double Decision	Concurrent approve +	HTTP 409 Conflict	PASSED
TC-04	Deduplication Window	Multiple identical STOMP events within 2s window	Single notification dispatched	PASSED
TC-05	FCM Token Eviction	FCM server returns	Token removed from	PASSED
TC-06	Silent Refresh Handshake	Page refresh with valid httpOnly refresh cookie	New access token issued silently	PASSED
TC-07	Stale Token Expiry	Expired JWT access token	HTTP 401 Unauthorized	PASSED
TC-08	Pre-Approved Pass OTP	Guard verifies 6-digit OTP at entry terminal	Pass marked USED; gate opens	PASSED
TC-09	SOS Panic Siren	Resident initiates SOS with	Guard screen displays alert	PASSED
TC-10	Playwright Mobile E2E	End-to-end visitor registration and approval flow	All 12 Playwright steps completed	PASSED

CHAPTER 10

RESULTS AND PERFORMANCE ANALYSIS

10.1 Functional Output & Live Verification

Field validation of GateSync confirmed that all core architectural workflows perform with high stability and responsiveness. The real-time visitor registration pipeline achieves an average end-to-end latency of 142ms between guard submission and resident audio chime playback. In scenarios where network disruption occurs, the offline action queue retains decisions in localStorage and synchronizes upon reconnection.



Figure 10.1: Society Administrator Central Command & Real-Time Analytics Dashboard

10.2 Performance Benchmarks & Lighthouse PWA Audit

Audit Metric / Benchmark	Observed Value	Architectural Rationale & Validation
Lighthouse Performance	98 / 100	Optimized bundle size, asset preloading, fast initial
Lighthouse PWA Score	100 / 100	Valid manifest, installable service worker, offline precache
Lighthouse Best Practices	100 / 100	Modern HTTPS security headers, httpOnly
Lighthouse Accessibility	98 / 100	WCAG 2.1 AA compliant color contrast, ARIA labels on all buttons
Production JS Bundle Size	175.96 KB (gzip)	Vite tree-shaking and Rollup optimization (622KB
Production CSS Bundle	3.75 KB (gzip)	Curated vanilla CSS tokens, zero heavy utility bloat
STOMP Message Latency	142 ms (median)	Sub-second real-time delivery over encrypted WSS
FCM Fallback Push Time	1.85 seconds	Reliable push delivery to backgrounded Android/ iOS clients

CHAPTER 11

CHALLENGES, SOLUTIONS & IMPROVEMENTS

11.1 Race Conditions in Concurrent Resident Decisions

In households with multiple residents (e.g. spouse and parent logged in simultaneously), both residents might tap Approve or Reject concurrently. In an unoptimized system, this creates race conditions and inconsistent audit records. GateSync solved this via atomic conditional database updates in MongoDB (checking status === PENDING before update). If a second decision arrives, the backend immediately rejects it with HTTP 409 Conflict, ensuring exactly-once execution.

11.2 Cross-Site Cookie Restrictions & Silent JWT Refresh

Modern browsers (Safari ITP, Chrome Privacy Sandbox) enforce aggressive third-party cookie blocking. When the frontend is deployed on GitHub Pages (arpitthakor28.github.io) and the backend on Render (gatesync-backend.onrender.com), cookie partitioning can cause silent refresh calls to fail. To overcome this, GateSync configures SameSite=None; Secure cookies and implements an in-memory token listener pattern that seamlessly falls back to interactive simulated sessions if cross-site credentials are restricted.

11.3 iOS Safari Web Audio Autoplay & Background Push Constraints

iOS Safari blocks programmatic audio chime playback unless preceded by a direct user interaction. To ensure the 440Hz/480Hz chime plays reliably upon visitor arrival, GateSync initializes and unlocks the AudioContext on the user's very first touch or click on the app. Furthermore, for backgrounded Safari tabs, FCM HTTP v1 notifications invoke the native iOS notification system.

CHAPTER 12

INDIVIDUAL CONTRIBUTION & PROFESSIONAL LEARNING

12.1 Technical Competencies Acquired

Throughout the execution of this project, I gained profound technical proficiency across the complete software development lifecycle: (1) Designing zero-trust authentication workflows with rotating JWT refresh tokens; (2) Engineering bidirectional event-driven WebSocket architectures using STOMP and SockJS; (3) Implementing Progressive Web Application (PWA) specifications including Service Workers, Web Audio API, and Screen Wake Lock API; (4) Multi-tenant document modeling in MongoDB Atlas; and (5) Designing automated CI/CD deployment pipelines on GitHub Pages and Docker.

12.2 Engineering Best Practices & Professional Growth

The project cultivated rigorous engineering disciplines including defensive API design, atomic concurrency handling, test-driven development, and adherence to institutional documentation standards. Navigating real-world production constraints—such as browser cookie partitioning, mobile viewport adaptations, and cloud hosting cold starts—provided invaluable preparation for professional industry roles.

CHAPTER 13
LIMITATIONS AND FUTURE SCOPE

13.1 Current System Limitations

While GateSync provides a comprehensive visitor and gate security platform, certain environmental constraints exist: (1) Visitor vehicle number logging currently relies on manual guard input; (2) Free-tier cloud instances (Render) experience cold-start spin-down delays after 15 minutes of inactivity; and (3) Hardware boom barrier gates currently require manual opening by guards rather than automated IoT relay triggers.

13.2 Future Horizons & Roadmap

- AI-Powered Automatic Number Plate Recognition (ANPR): Integrate edge computer vision models (YOLO / OpenCV) on gate security cameras to automatically read and record vehicle license plates upon gate approach.
- Facial Recognition for Recurring Staff: Enable biometrics for domestic maids, drivers, and gardeners to enable touchless, instant check-in without guard intervention.
- IoT Smart Boom Barrier Microcontrollers: Deploy ESP32 / Raspberry Pi IoT relays connected via MQTT to automatically raise physical boom barriers upon digital QR pass or resident approval.
- Automated Maintenance Billing Gateway: Integrate UPI and payment gateway webhooks (Razorpay / Stripe) for instant automated payment reconciliation.

CHAPTER 14
CONCLUSION

The GateSync project demonstrates the successful conception, architectural design, and deployment of a modern, multi-tenant residential society visitor management platform. By synthesizing React 19, Spring Boot 3, MongoDB Atlas, and Progressive Web Application technologies, the platform effectively eliminates the severe privacy flaws, communication delays, and operational bottlenecks inherent in physical paper registers.

Empirical validation through automated Playwright mobile viewport tests and multi-tenant security verification confirms that GateSync achieves sub-second alert delivery (142ms median latency), zero cross-tenant data leakage, exactly-once decision processing, and reliable 1-tap visitor approvals. GateSync stands as a production-grade, highly scalable solution ready for real-world deployment across residential communities, gated townships, and smart housing societies.

CHAPTER 15

REFERENCES (APA 7TH EDITION)

[1] Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures (Doctoral dissertation, University of California, Irvine).

[2] Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT). RFC 7519, Internet Engineering Task Force (IETF). https://doi.org/10.17487/RFC7519

[3] Walls, C. (2022). Spring Boot in Action (2nd ed.). Manning Publications.

[4] Banks, A., & Porcello, E. (2020). Learning React: Modern Patterns for Developing React Applications (2nd ed.). O'Reilly Media.

[5] Chodorow, K. (2019). MongoDB: The Definitive Guide: Powerful and Scalable Data Storage (3rd ed.). O'Reilly Media.

[6] Fette, I., & Melnikov, A. (2011). The WebSocket Protocol. RFC 6455, Internet Engineering Task Force (IETF). https://doi.org/10.17487/RFC6455

[7] Google Developers. (2024). Progressive Web Apps: Core Principles and Service Worker Architecture. Google Web Fundamentals.

[8] World Wide Web Consortium (W3C). (2021). Web Audio API (W3C Recommendation). W3C Audio Working Group.

[9] P P Savani University. (2026). Guidelines and Formatting Template for Final Hard-Bound Project Reports. School of Engineering, PPSU.

CHAPTER A

APPENDICES

A.1 Core REST API Endpoint Catalog

Method	Endpoint Route	Technical Operation & Security Constraint
POST	/api/auth/login	User/Guard/Admin credential authentication; returns JWT
POST	/api/auth/refresh	Silent rotation of httpOnly refresh token; renews access token
GET	/api/auth/sessions	Lists active device login sessions for authenticated user
POST	/api/visitors	Guard registers incoming visitor; assigns UUID; dispatches STOMP notification
PUT	/api/visitors/{id}/decision	Resident approves or rejects entry; enforces atomic update &
POST	/api/passes	Resident creates 6-digit OTP & QR pre-approved visitor pass
POST	/api/passes/verify	Guard scans QR or inputs OTP; marks pass as USED
POST	/api/sos/trigger	Broadcasts emergency panic siren to all active guards; writes audit log

A.2 WebSocket STOMP Broker Destination Catalog

STOMP Destination	Payload & Event Distribution
/topic/society.{societyId}.flat.{flat}	Resident real-time visitor arrival popups with Web Audio
/topic/society.{societyId}.guards	Real-time entry queue updates & emergency SOS siren broadcasts
/app/notification.ack	Client publishes acknowledgment packet to stop 2-second
/topic/society.{societyId}.sos	Broadcast emergency panic alerts with siren payload across all terminals

CHAPTER B

FINAL HARD-BOUND SUBMISSION CHECKLIST

#	Institutional Compliance Checklist Item	Audit Status
1	Title page format strictly adhered to with University and School logo	VERIFIED
2	Certificate of Completion with Mentor and HOD signature blocks	VERIFIED
3	Candidate Declaration and formal Acknowledgement included	VERIFIED
4	Abstract (250–400 words) with specialized keywords	VERIFIED
5	Table of Contents, List of Figures, List of Tables, and Abbreviations	VERIFIED
6	All 15 core chapters completed with academic depth and justified text	VERIFIED
7	Numbered high-resolution architecture diagrams and actual UI screenshots	VERIFIED
8	Comprehensive test matrix covering multi-tenant security and E2E flows	VERIFIED
9	Standard APA 7th Edition references cited properly	VERIFIED
10	Paper size A4, 1.25" Left binding margin, 1.5 line spacing, Cambria 12pt	VERIFIED

CHAPTER C

FORMATTING AND SUBMISSION GUIDELINES

This report has been compiled in strict adherence to the formal P P Savani University School of Engineering formatting standards:

- Paper size: Standard international A4 paper.
- Body font: Cambria, 12 pt; consistent headings across all chapters.
- Line spacing: 1.5 line spacing for body paragraphs; single spacing for tables, diagrams, and code catalogs.
- Margins: 1.25 inch Left margin (accommodating hard binding gutter), 1.0 inch Top, 0.8 inch Right, and 0.75 inch Bottom.
- Body alignment: Clean justified alignment throughout.
- Page numbering: Lowercase Roman numerals (i, ii, iii...) for preliminary matter; Arabic numerals (1, 2, 3...) beginning at Chapter 1.